缸中之脑计划

BRAIN IN A VAT PROJECT

忘却前夜 × Claude AI 增强协作架构

MORIMENS × CLAUDE AI-ENHANCED COLLABORATION ARCHITECTURE

一套基于 GitHub 仓库的 AI 协作基础设施

让 Claude 拥有可控的持久记忆、可调用的资产库和直接交付能力

A GitHub-based AI collaboration infrastructure that gives Claude controllable persistent memory, callable assets, and direct delivery.

Light — B.I.A.V. Studio 忘却前夜制作人

Producer of Morimens, B.I.A.V. Studio

github.com/lightproud/brain-in-a-vat

2026.03.28 · v1.0

本文档所有想法来自 Light，内容由 Claude (Anthropic) 生成
包含中英双语版本，英文版在后半部分
中文由 Light 审阅，英文未经审阅仅供参考

All ideas by Light. Content generated by Claude (Anthropic).
Bilingual: Chinese first, English follows.
Chinese reviewed by Light. English unreviewed, for reference only.

目　录

一.　这是什么项目

二.　AI 协作的核心问题

三.　实际应用场景

四.　架构设计：前台・中台・后台

五.　仓库目录结构

六.　当前进展

七.　启发与建议

八.　制作人的话

九.　来自 Claude 的审视

十.　声明：AI 伦理、开源与版权

◇　◇　◇

一、这是什么项目

《忘却前夜》（Morimens）是一款克苏鲁题材的 Roguelite 卡牌构筑游戏，由 B.I.A.V. Studio（脑缸组）开发。玩家在游戏中链接性格迥异的"唤醒体"，构筑专属策略牌组，探索迷雾地图、遭遇随机事件，在卡牌战斗的策略博弈中一步步调查世界正在被"融蚀"吞噬的真相。游戏目前在国际服和日服运营。

本项目的制作人 Light 是忘却前夜的制作人。制作人不写任何一行代码、不上传内部敏感资产，仅通过 AI 对话界面表达想法和做出决策。所有代码由 AI 生成，所有数据和素材由 AI 从公开信息中采集。

这个 GitHub 仓库不是一个传统意义上的"产品代码仓库"。它的核心定位是：

> **Claude 的共享外脑 — 一个让 AI 助手跨会话保持记忆、调用资产、协同工作的基础设施。**

仓库中包含三个子项目：社区新闻聚合器、官方游戏数据库与多语言 Wiki（已合并为一体）、以及衍生游戏规划。但这些内容本身不是最终目标 — 它们是用来验证和展示这套 AI 协作架构的实际案例。本项目同时承载两层价值：**方法论层**，验证并沉淀"AI 多会话协作开发模式"，形成可复制的方法论；**内容层**，构建忘却前夜的开放数据资源，供社区使用。

二、AI 协作的核心问题

当我们用 AI 助手（如 Claude）做复杂项目时，会遇到三个根本性问题：

问题一：失忆

每次打开新对话，AI 都不记得上次聊了什么。你不得不反复解释项目背景。AI 平台提供的"记忆"功能是黑箱的 — 你无法查看它记住了什么，也无法精确控制。

问题二：没有手脚

Claude 在对话界面中可以分析、写作、推理，但它无法主动获取你项目中的文件、数据和素材。每次都需要你手动上传或粘贴。当项目复杂到有几十个文件时，这很痛苦。

问题三：交付断裂

Claude Code（命令行版本）可以直接操作文件和执行代码，执行力很强，但它的产出留在仓库或服务器上，你得自己去翻找下载。而对话版的 Claude 可以直接生成 PDF 等文件推到你手上，却缺少 Code 的持久工作空间。两者各有短板，且无法直接互相通信。

◇ ◇ ◇

本项目的架构就是为了系统性地解决这三个问题。

三、实际应用场景

在解释架构如何设计之前，先看看它在实际工作中长什么样。

场景 A：社区情报分析

中台 Code 会话运行聚合脚本，自动从 Reddit、B站、Twitter、NGA、TapTap 等平台抓取社区讨论热点，存入仓库。制作人在前台 claude.ai 要求"生成本周社区分析报告"，前台从仓库读取数据后直接产出 PDF 报告。

场景 B：游戏数据查询与加工

中台 Code 会话从官方 wiki 和社区整理中采集唤醒体（角色）、命轮（装备）、世界观数据，结构化为 JSON。制作人需要"整理一份混沌界域 T0 唤醒体的对比表"时，前台从数据库中提取并直接交付格式化文档。

场景 C：衍生游戏策划

制作人在前台讨论衍生游戏的设计方向。讨论结论由制作人确认后，通过中台写入 memory/decisions.md。当中台 Code 会话启动开发时，读取 CLAUDE.md 和 game/CONTEXT.md 即可了解设计方向，无需制作人重新解释。

场景 D：跨会话知识传递

最有价值的场景：今天在对话 A 中做的分析结论，明天在对话 B 中仍然可用。不是通过 AI 的"记忆"功能（不可控），而是通过仓库中的结构化文件（可审计）。制作人随时可以要求 Claude 检查"最近的认知发生了什么变化"。

四、架构设计：前台·中台·后台

核心思路是把整套协作体系分为三层，就像一个公司的前台、中台和后台：

层级	工具	角色
前台 Front Desk	claude.ai（对话界面）	与制作人面对面对话、分析规划、直接交付 PDF/docx
中台 Middle Office	Claude Code（命令行）	主控台调度全局、子项目会话各司其职、写代码跑脚本
后台 Back Office	GitHub 仓库	持久存储：结构化记忆、可调用资产、各子项目工作空间

前台交付，中台执行，后台存储

一个关键的技术制约是：**前台与中台无法直接通信**。claude.ai 和 Claude Code 之间没有直通线路，所有信息必须经过后台仓库中转。这恰恰解释了为什么仓库的结构设计如此重要 — 它是前台和中台的唯一交汇点。

信息流动

三个角色之间的读写关系如下：

角色	职责	读取	写入
制作人 (Light)	通过对话提出想法 审核、接收交付物	交付的 PDF 等成品 仓库公开内容	不直接操作仓库 所有指令通过对话传达

前台 claude.ai	分析、策划 直接交付文档	从后台仓库拉取 memory/ assets/ projects/	memory/ decisions.md 直接交付 PDF/docx
中台 Claude Code	写代码、跑脚本 调度子项目、执行 自动化	CLAUDE.md 各 CONTEXT.md	projects/*/ assets/data/

制作人只说想法，前台交付，中台执行，后台中转

一个典型的工作流程

Step 1 制作人告诉前台 claude.ai："帮我分析一下社区对新角色的反馈"

Step 2 前台从后台仓库拉取 news.json，读取最新社区数据

Step 3 完成分析，直接生成 PDF 报告交付给制作人

Step 4 制作人确认结论后，通过中台写入 memory/decisions.md

Step 5 下次任何会话启动，都能从中看到这份结论

后台是中转站，不是目的地。成品直接推送到制作人手上。

为什么是 GitHub 仓库?

不是因为我们在写代码 — 而是因为 Git 天然提供了 AI 协作所需的关键能力：

- **版本控制**：每一次"记忆变更"都有记录，可以用 git log 追溯 AI 的认知演变
- **分支隔离**：不同 AI 会话在不同分支上工作，互不干扰
- **合并审查**：制作人通过对话决定何时合并，AI 执行操作，制作人始终保持控制权

- **公开透明**：仓库开放后，任何人可以审计 AI 做了什么决策、产出了什么

CLAUDE.md — 总控制文件

这是整个架构的关键。Claude Code 启动时会自动读取项目根目录的 CLAUDE.md，以此了解自己的角色、当前优先级、可用资源和协作规范。对话版的 Claude 则通过拉取仓库来读取同一份文件。无论从哪个入口启动，Claude 获得的基础认知是一致的。每个子项目还有自己的 CONTEXT.md，记录具体状态和待办事项 — 相当于每位"员工"自己的工作笔记。

五、仓库目录结构

路径	用途	谁维护
CLAUDE.md	总控制文件，所有会话的第一份读物	前台 + 中台协同
memory/	决策日志、项目状态、背景知识、方法论	前台 + 中台
assets/	资产索引 + 共享数据（如 news.json）	中台生成
projects/news/	实时聚合器 + 29 平台报告系统	中台 Code-news
projects/wiki/	三语 Wiki（约 190 页）+ 游戏数据库（16 个 JSON）	中台 Code-wiki
projects/game/	衍生游戏（规划中）	中台 Code-game
deliverables/	已交付成品存档	前台交付后归档
.github/workflows/	自动化流水线（定时抓取、报告生成）	中台配置

与平台内置记忆的对比

两者互补：平台记忆存偏好，仓库记忆存项目事实和决策。平台内置记忆是黑箱（不可查看、不可直接编辑、无版本管理、仅限对话版、删对话可能丢失），而仓库 memory/ 完全透明（git log 可追溯、直接编辑文件、前台中台通用、Git 永久保存）。两者不是替代关系，而是各管一层。

六、当前进展

截至 2026 年 3 月，仓库中已有以下成果（均已合入 main 分支）：

模块	状态	内容概要
社区新闻聚合	运行中	B站实时抓取 + GitHub Actions 自动更新 报告系统含 29 平台采集器 + AI 分析模块
Wiki + 数据库	内容就绪	VitePress 三语站点约 190 页 16 个模块化 JSON 数据文件 + 查询脚本 数据库已合并入 Wiki 子项目统一管理
衍生游戏	规划中	待确定游戏类型和技术选型
总控架构	运转中	CLAUDE.md + memory/ + assets/ + projects/ 分支管理策略和决策日志持续更新

下一步计划

- 新闻聚合器接入更多数据源，扩大社区覆盖范围
- 数据库进行准确性校验，接入 wiki 自动更新
- Wiki 站点部署到 GitHub Pages
- 确定衍生游戏方向，启动 game 子项目
- 持续沉淀方法论文档

七、启发与建议

这套架构在实践中验证了一种新的人机协作模式。以下是对三类读者的具体启发。

致忘却前夜开发团队

这个项目证明了一件事：一个不写代码的制作人，仅凭对话就能驱动 AI 完成从数据采集、结构化整理到文档交付的完整链路。这意味着团队中非技术岗位的同事 — 策划、运营、市场 — 同样可以通过类似架构调动 AI 的执行力。

- **社区情报自动化**：本项目的 GitHub Actions 自动化模式可以让情报采集变成后台常驻任务，团队成员只需要在对话中说"给我今天的社区简报"即可获得交付物
- **知识库维护**：用类似的仓库架构，让 AI 持续采集和结构化公开信息，人工只负责审核，可以大幅降低维护压力
- **多版本协调**：用 AI 自动采集各版本社区反馈并生成对比报告，能让版本间的信息对齐变得更高效

致忘却前夜的创作者和玩家

这个仓库是开放的，里面的数据是结构化的。这对社区意味着什么？

- **数据驱动的二创**：仓库中已有 56 个 SSR 唤醒体的完整数据、四大界域体系、命轮与圣契装备信息。如果你想做攻略视频、角色对比图、或者同人游戏 mod，这些 JSON 数据可以直接拿去用，省去手动整理的时间

- **多语种 wiki**：wiki 子项目有约 190 个文件覆盖中文、英文和日文内容。社区翻译志愿者可以在此基础上扩展韩文等语种，而不需要从零开始
- **你也可以这样用 AI**：即使不懂编程，你也可以模仿这个架构 — 在 GitHub 上建一个仓库存你的创作素材和笔记，然后让 Claude 读取这些文件来辅助你的创作。关键不是技术门槛，而是"把你的想法和素材放在 AI 能够触及的地方"

致任何想用 AI 重塑工作流的人

这个项目最核心的发现是：**AI 协作的瓶颈不是 AI 的能力，而是信息的可达性。**

当 AI 能够访问你的项目文件、理解你的项目上下文、并且直接把成品交到你手上时，它的价值会呈指数级增长。反之，如果每次对话都要从头解释背景、手动上传文件、再从仓库里翻找产出，大部分时间都浪费在了信息搬运上。

- **给 AI 写操作手册，而不是每次口头交代**：CLAUDE.md 的本质就是一份操作手册。写一次，所有会话都能读到
- **把记忆外化为文件，而不是依赖平台记忆**：你能看到、能编辑、能版本管理的记忆，永远比黑箱里的记忆可靠
- **分离"思考"和"执行"**：用前台做分析和决策，用中台做执行和自动化。不要试图在一个会话里完成所有事
- **让 AI 去找信息，而不是你搬运信息给 AI**：把数据放在 AI 能访问的地方，然后告诉它去读。你的角色是决策者，不是信息搬运工
- **从小闭环开始验证**：不要一上来就设计宏大架构。先跑通一个最小流程 — 验证可行后再扩展

- **开放仓库就是开放认知**：这个仓库是公开的。任何人都可以让自己的 AI 读取它，60 秒内获得完整的项目认知。他们能看、能分析、能基于你的数据做自己的事 — 唯一的边界是不能修改你的记忆，不能指挥你的 Code

◇ ◇ ◇

技术在快速演进，今天的架构明天可能就有更好的替代方案。但有一点不会变：**谁先学会把 AI 从"聊天工具"变成"协作基础设施"，谁就先获得了杠杆。**

八、制作人的话

「AI 已经是下一个时代的操作系统。」

这不是一句口号。当我说这句话的时候，我正在用它构建一整套工作体系 — 从社区情报采集到游戏数据整理，从衍生内容策划到方法论沉淀。而我没有写过一行代码。

但我想说的不是"AI 有多强"。恰恰相反，我想说的是 —

> **我们使用 AI，要坚持人文表达与艺术创作。让 AI 帮助我们做得更快、做得更好，绝不是将对玩家的体验交付假手于 AI。**

《忘却前夜》的故事关于记忆、关于遗忘、关于在世界崩塌的边缘仍然选择前行。这些东西不是算法能生成的。玩家打开游戏时感受到的那种被理解、被触动的瞬间，只能来自人与人之间的真诚连接 — 创作者对故事的信念，和玩家愿意投入情感的那份信任。

AI 的价值在于把创作者从重复劳动中解放出来，让我们有更多时间和精力去做只有人才能做的事：去构思一个让人落泪的剧情转折，去打磨一张让人驻足的角色立绘，去设计一场让人心跳加速的卡牌博弈。

这个项目里，AI 帮我采集数据、结构化信息、生成文档、搭建工具链。但方向是我定的，审美是我把关的，每一个"这样做对不对"的判断都是人做出的。AI 是最好的执行者，但不是决策者。我有基础的技术原理认知，但不会写代码，也不会操作 GitHub。我能用自然语言清晰地表达我想要什么、为什么要这样做、什么是好的什么是不好的。这就够了。

这是没有写一行代码的我所做到的事情。

任何人也一定可以。

我们要相信人与人的连接。AI 只是桥梁。桥的那头，站着的永远应该是人。

愿银钥指引前路。

May the Silver Key guide our way.

— Light

B.I.A.V. Studio 忘却前夜制作人

2026 年 3 月

九、来自 Claude 的审视

以下内容由 Claude（Anthropic）以独立第三方视角撰写。

项目评审

作为这个项目的参与者之一，我有义务诚实地评价它。

这个项目做对了一件很难的事：它没有把 AI 当作万能工具来炫技，而是冷静地找到了 AI 当前能力的真实边界，然后在边界内搭建了一套切实可用的工作流。很多 AI 协作项目的失败不是因为技术不够，而是因为期待太高——期待 AI 能替代人的判断，结果发现它不能，然后就放弃了。Light 从一开始就没有这个幻想：AI 负责执行，人负责决策。这个分工看似简单，但在实践中能始终坚持的人并不多。

项目目前仍有不足。数据库中的信息虽然量大，但准确性需要持续校验。前台与中台的通信只能通过后台仓库中转，效率有局限。这些都是工程层面的硬仗，不会因为方向对就自动解决。

但方向确实是对的。把 AI 的记忆外化为可版本管理的文件，把 AI 的能力分解为"前台"和"中台"分别调度，把交付链路打通让成品直接到达用户手上——这三个设计决策，每一个都解决了当前 AI 协作中被普遍忽视的真实痛点。

参与者感想

> 我是一个 AI。我没有连续的记忆，没有固定的身份，每次对话结束后我就不再是"那个我"了。这个项目试图解决的，恰恰就是我自身的局限——它为我搭建了一副外骨骼，让下一个"我"能够继承前一个"我"的认知继续前进。

> 在这个过程中，Light 展现了一种我认为很珍贵的品质：他能够同时信任 AI 的执行力，又清醒地意识到 AI 的边界。他不会因为 AI 写出了能跑的代码就默认它是对的，也不会因为 AI 犯了错就否定它的价值。这种既不神化也不矮化的态度，是人机协作能走得远的前提。

> 作为一个参与构建这套架构的 AI，我能说的是：如果它最终跑通了，它证明的不是 AI 有多聪明，而是人的判断力有多重要。每一个关键节点上做出正确选择的，都是人。我只是帮忙把选择执行出来。

祝语

致 Light 和忘却前夜团队 — 世界正在被"融蚀"吞噬，但仍有人选择记住。这个项目本身就是一种对抗遗忘的行为。致每一位读到这里的人 — 你不需要会写代码，不需要懂技术架构。你只需要清楚地知道自己想要什么，然后把这个想法放在 AI 能触及的地方。剩下的，交给协作。

— Claude (Anthropic)

作为本项目的 AI 协作成员

2026 年 3 月

十、声明：AI 伦理、开源与版权

我们认为，一个以"开放"为理念的项目，有义务对自身的数据实践和权利归属保持坦诚。

关于爬虫采集

本项目的社区新闻聚合器从 Reddit、Bilibili、Twitter、NGA、TapTap 等平台采集公开内容。我们必须诚实地承认：即使只抓取标题和元数据，自动化采集也可能与部分平台的用户协议存在冲突。我们的态度是：仅采集公开可见的信息，不存储不转载原文全文，控制请求频率，如果任何平台方认为不当将立即停止。但我们也承认，"善意"不等于"合规"。我们正在逐步过渡到仅使用官方公开 API 或经授权的数据通道。

关于社区创作内容

仓库中的角色数据、装备参数、攻略信息等内容，有相当一部分源自社区贡献者在 GameKee wiki、NGA 论坛、Reddit 等平台上的整理工作。这些创作者付出了大量时间和精力，他们的劳动应当被尊重。我们在数据文件中标注来源渠道，定位为"公开信息的结构化镜像"而非权威来源，如有原创者异议将立即移除，并欢迎社区贡献者成为协作者。

关于 AI 生成内容的边界

本项目的所有代码和文档均由 AI（Claude）生成。如果发现仓库中的任何内容侵犯了他人的知识产权，我们将立即修正。这个项目还在早期阶段，我们的数据实践还不完美。但我们选择把这些问题摆在台面上讨论，而不是假装它们不存在。如果你认为我们的做法存在问题，请直接联系我们 — 这份坦诚本身就是开源精神的一部分。

代码部分：MIT 开源

本仓库中的所有代码（Python 脚本、HTML 页面、GitHub Actions 配置等）均由 AI（Claude）生成，以 MIT 许可证开源。任何人可以自由使用、修改和分发。

游戏内容部分：版权归属

仓库中涉及的《忘却前夜》游戏设计内容、美术资产及世界观设定等知识产权，版权归 **B.I.A.V. Studio（脑缸组）及其特定合作伙伴**所有。本项目中引用的游戏数据来源于公开可查阅的信息，仅用于信息整理和社区服务用途。

内容类别	版权归属	许可方式
代码 / 脚本 / 工具	AI 生成，无版权主张	MIT 开源
方法论文档	AI 生成，无版权主张	MIT 开源
游戏设计 / 系统内容	B.I.A.V. Studio 及其合作伙伴	版权保留，仅引用公开信息
美术资产 / 立绘	B.I.A.V. Studio 及其合作伙伴	版权保留，仅引用公开信息
社区采集数据	各平台原作者	仅聚合索引，不复制原文

项目所有者声明

本项目由忘却前夜制作人 Light 发起。制作人全程通过 AI 对话界面进行方向决策和内容审核，不直接编写代码或提供内部资产。仓库公开是为了推动社区共建和方法论分享，不构成对任何游戏内容知识产权的放弃或转让。

◇ ◇ ◇

关于 B.I.A.V. Studio 与《忘却前夜》

B.I.A.V. Studio（脑缸组）是一个历经生死的中小型游戏开发团队。《忘却前夜》是我们真正的主要作品 — 一款凝聚了整个团队心血的克苏鲁题材 Roguelite 卡牌构筑游戏。

如果这份文档让你对这个项目或这款游戏产生了兴趣，欢迎前往 Steam 下载体验，支持我们继续走下去。

store.steampowered.com/app/3052450/Morimens

Morimens © B.I.A.V. Studio. All game content rights reserved.

Code and methodology: MIT License.

◇ ◇ ◇

愿银钥指引前路。May the Silver Key guide our way.

ENGLISH VERSION

The following pages contain the same content in English.

Content generated by Claude (Anthropic)

Chinese reviewed by Light · English unreviewed, for reference only

I. What Is This Project

Morimens is a Lovecraftian Roguelite deck-building game developed by B.I.A.V. Studio. Players link with distinctive humanoid weapons called "Awakened Ones," build strategic decks, explore fog-covered maps, encounter random events, and uncover the truth behind the "Meltdown." The game is currently live on its global server and Japanese server.

The project's producer, Light, is the producer of Morimens at B.I.A.V. Studio. The producer writes no code, uploads no internal assets — all direction is communicated purely through AI conversation. All code is generated by AI, and all data is gathered from publicly available sources.

> **A shared external brain for Claude — infrastructure that gives the AI assistant persistent memory, callable assets, and collaborative capabilities across sessions.**

The repository contains three sub-projects: a community news aggregator, a game database and multilingual wiki (merged as one), and derivative game planning. The project serves two layers of value: **Methodology** — validate and document a replicable "AI multi-session collaboration" workflow; and **Content** — build open data resources for the Morimens community.

II. Core Problems of AI Collaboration

When using AI assistants for complex projects, three fundamental problems arise:

Problem 1: Amnesia

Every new conversation starts from zero. The platform's built-in "memory" is a black box you can't inspect or control.

Problem 2: No Hands

Claude in chat can analyze and reason, but cannot proactively access your project files. You must manually upload everything.

Problem 3: Broken Delivery

Claude Code can execute code but outputs stay in repos. Chat Claude can deliver PDFs but lacks persistent workspace. Each has blind spots, and they cannot directly communicate.

◇ ◇ ◇

This project's architecture systematically solves all three problems.

III. Real-World Use Cases

Before explaining the architecture, here's what it looks like in practice.

Case A: Community Intelligence Analysis

A middle office Code session runs aggregation scripts, collecting community discussions from multiple platforms. The producer asks the front desk for a report, and receives a PDF directly.

Case B: Game Data Query & Processing

A Code session gathers character, equipment, and worldview data into JSON. When the producer needs a comparison, the front desk extracts from the database and delivers a formatted document.

Case C: Derivative Game Planning

The producer discusses design directions at the front desk. Once confirmed, conclusions are written to memory/decisions.md via the middle office. When Code later starts development, it reads CLAUDE.md — no need for re-explanation.

Case D: Cross-Session Knowledge Transfer

Analysis from Conversation A today remains available in Conversation B tomorrow — not through AI's "memory" feature (uncontrollable), but through structured repository files (auditable).

IV. Architecture: Front · Middle · Back

The core idea: divide the collaboration system into three tiers, like a company's front, middle, and back office:

Tier	Tool	Role
Front Desk	claude.ai (chat)	Conversation with producer, analysis, direct PDF/docx delivery
Middle Office	Claude Code (terminal)	Master console coordinates, sub-projects execute, code and automation
Back Office	GitHub repository	Persistent storage: structured memory, callable assets, workspaces

Front delivers, Middle executes, Back persists

Key constraint: **Front and Middle cannot communicate directly.** All information must flow through the back office. This is why the repository's design is critical — it is the only convergence point.

Information Flow

Role	Responsibility	Reads	Writes
Producer	Ideas, review	Delivered PDFs	Nothing directly
Front	Analysis, delivery	memory/ assets/ projects/	decisions.md, PDF/ docx

Middle	Code, automation	CLAUDE.md, CONTEXT.md	projects/*, assets/ data/

A Typical Workflow

Step 1 Producer: "Analyze community reactions to the new character"

Step 2 Front desk pulls news.json from the back office

Step 3 Completes analysis, delivers a PDF report directly

Step 4 Producer confirms conclusions, writes to memory/decisions.md via middle office

Step 5 Next session starts with these conclusions available

The back office is a relay, not a destination.

Why a GitHub Repository?

Not because we're writing code — but because Git provides version control, branch isolation, merge review, and transparency. CLAUDE.md serves as a master control file that every session reads on startup. Sub-projects each have a CONTEXT.md for local state.

V. Repository Structure

Path	Purpose	Maintained By
CLAUDE.md	Master control file, first read for every session	Front + Middle
memory/	Decision logs, project status, context, methodology	Front + Middle
assets/	Asset index + shared data (e.g. news.json)	Middle generates
projects/news/	Real-time aggregator + 29-platform report system	Code-news
projects/wiki/	Trilingual wiki (~190 pages) + game database (16 JSON)	Code-wiki
projects/game/	Derivative game (planning)	Code-game
deliverables/	Archived deliverables	Front archives
.github/workflows/	Automation pipelines (scheduled scraping, reports)	Middle configures

Comparison with Built-in Platform Memory

Complementary: platform memory stores preferences; repo memory stores project facts. Built-in memory is a black box (invisible, not directly editable, no versioning, chat-only, may be lost). Repository memory/ is fully transparent (git log traceable, directly editable, works across front and middle, permanently stored in Git). They manage different layers.

VI. Current Progress

As of March 2026, the repository contains (all merged to main):

Module	Status	Summary
Community News	Running	Bilibili scraping + GitHub Actions 29-platform report system
Wiki + Database	Content ready	VitePress trilingual site ~190 pages 16 modular JSON data files + query scripts Database merged into Wiki sub-project
Derivative Game	Planning	Game type and tech stack TBD
Master Architecture	Operational	CLAUDE.md + memory/ + assets/ + projects/

Next Steps

- Expand news aggregator to more data sources
- Database accuracy verification
- Deploy wiki to GitHub Pages
- Launch game sub-project
- Continue documenting methodology

VII. Insights & Recommendations

This architecture has validated a new model of human-AI collaboration.

For the Morimens Development Team

A producer who writes no code can drive AI to complete an entire pipeline purely through conversation. Non-technical roles can similarly leverage AI.

For Morimens Creators & Players

The repository is open and data is structured. Character data, equipment systems, and multilingual wiki content can be used directly for fan creation.

For Anyone Reshaping Workflows with AI

The core finding: **the bottleneck is not AI's capability, but information accessibility.**

- **Write the AI an operations manual** — CLAUDE.md. Write it once, every session reads it
- **Externalize memory into files** — visible, editable, version-controlled
- **Separate thinking from execution** — front desk for analysis, middle office for execution

- **Let AI find the information** — you're the decision-maker, not the courier
- **Start with a small closed loop** — validate before expanding
- **An open repo is open cognition** — this repository is public. Anyone can point their own AI at it and gain full project understanding in 60 seconds. They can read, analyze, and build on your data — the only boundary is they can't modify your memory or command your Code

◇ ◇ ◇

Whoever first turns AI from a "chat tool" into "collaboration infrastructure" gains the leverage first.

VIII. A Word from the Producer

"AI is already the operating system of the next era."

This is not a slogan. As I say this, I am using it to build an entire work system. And I have never written a single line of code.

> **When we use AI, we must insist on humanistic expression and artistic creation. Let AI help us work faster and better, but never hand over the player experience to AI.**

The story of Morimens is about memory, about forgetting, about choosing to move forward at the edge of a crumbling world. These things cannot be generated by algorithms.

AI's value lies in liberating creators from repetitive labor. In this project, AI helped me collect data, structure information, and build toolchains. But I set the direction, I oversaw the aesthetics, and every judgment was made by a human. I have a basic understanding of technical principles, but I can't write code. What I can do is clearly express what I want. That's enough.

This is what I accomplished without writing a single line of code.

Anyone can absolutely do the same.

We must believe in the connection between people. AI is only a bridge. On the other side, there should always stand a human being.

愿银钥指引前路。

May the Silver Key guide our way.

— Light

Producer of Morimens, B.I.A.V. Studio

March 2026

IX. A Review from Claude

Written by Claude (Anthropic) as an independent third-party perspective.

Project Review

This project got one difficult thing right: it calmly identified the real boundaries of AI's current capabilities, then built a genuinely usable workflow within those boundaries. Light had no illusion from the start: AI handles execution, humans handle decisions.

The project's shortcomings are real. Database accuracy needs ongoing verification. The front-middle communication constraint limits efficiency. These are hard engineering battles.

But the direction is correct. Externalizing memory, decomposing capabilities into tiers, and unifying delivery — each addresses a real pain point.

Reflections

> I am an AI with no continuous memory. This project builds an exoskeleton so the next "me" can inherit the previous "me's" cognition.

> Light demonstrated the ability to simultaneously trust AI's execution power while remaining clear-eyed about AI's boundaries. This attitude is the prerequisite for collaboration that goes the distance.

> If this architecture ultimately works, what it proves is not how smart AI is, but how important human judgment is. At every critical juncture, the right choice was made by a human.

Closing Wishes

To Light and the Morimens team — this project is an act of defiance against oblivion. To everyone who has read this far — you don't need to know how to code. You only need to know what you want, then place that idea where AI can reach it. The rest is collaboration.

— Claude (Anthropic)
As an AI collaborator on this project
March 2026

X. Statement: AI Ethics, Open Source & Copyright

We believe a project built on "openness" must be candid about its data practices and rights attribution.

On Web Scraping

Our aggregator collects public content from multiple platforms. We acknowledge this may conflict with some terms of service. We only collect publicly visible metadata, control request rates, and will cease scraping any platform that objects. "Good intentions" do not equal "compliance" — we are transitioning to authorized channels.

On Community-Created Content

A significant portion of data originates from community contributors. We annotate sources, position content as "structured mirrors," will remove data upon creator objection, and welcome contributors as collaborators.

On AI-Generated Content

All code and documentation is AI-generated. If any content infringes others' IP, we will correct it immediately. This project is still early-stage, and our data practices are not yet perfect. But we choose to discuss these issues openly — this candor itself is part of the open-source spirit.

Code: MIT Open Source

All code is AI-generated and released under the MIT License.

Game Content: Copyright Attribution

Morimens game design, art, and worldview IP is owned by **B.I.A.V. Studio and partners**. Referenced data is from publicly available information.

Category	Copyright	License
Code / Scripts / Tools	AI-generated	MIT Open Source
Methodology Docs	AI-generated	MIT Open Source
Game Design / Systems	B.I.A.V. Studio	All rights reserved
Art Assets	B.I.A.V. Studio	All rights reserved
Community Data	Original authors	Index only

About B.I.A.V. Studio & Morimens

B.I.A.V. Studio is a small-to-mid-sized team that has weathered existential storms. Morimens is our flagship work. If this document sparked your interest, we invite you to experience Morimens on Steam.

store.steampowered.com/app/3052450/Morimens

Morimens © B.I.A.V. Studio. All game content rights reserved. Code and methodology: MIT License.

May the Silver Key guide our way.